Two alternative 215-state ZFC consistency machines

Re-encoding the source and admitting a nonzero initial counter

Research contribution for Vladimir Reshetnikov's ProveIt project

21 October 2026 UTC

Abstract

We give two binary, single-tape, blank-start machines with 215 working states whose nonhalting is equivalent to the consistency of ZFC. The primary construction is a source re-encoding. This is an alternative construction, not a 214-state result. Its transition graph is not a relabeling of the three compared 215-state baselines. Relative to the primary baseline, it changes 17 transition rows and 4,519 source instruction addresses, while retaining the same 4,751 primitive operations and their complete guarded ordinal control graph. The number of addresses containing six consecutive zero bits decreases from 37 to 33; the source DAG remains at 202 nodes.

The important additional proof obligation is initialization. The new machine reaches its genuine source root after 716,307 transitions with next-candidate register 4 and additive counter 1, rather than the old zero-counter seed. We prove that an arbitrary natural initial counter is harmless: command soundness already permits it, and a cofinal family of refutation programs begins by resetting it. Fresh unbounded runtime verification establishes all 6,176 guarded primitive cases for this literal table. The resulting proof is ordinary mathematics formalizable in ZFC, not a newly kernel-checked Lean or Rocq theorem. The calculus itself is inherited unchanged; the primary contribution is a materially different physical encoding and a more flexible initialization theorem. A second table gives a distinct physical allocation of the recently published sixth-axiom sampler source, together with its adjusted strong compiler and separate runtime and blank-start verification.

Contents

1 Result, distinction, and scope **2**

2 Equivalent source encoding and physical verification **3**

3 The numerical calculus and its consistency calibration **3**

4 Inherited all-natural calibration of the local-input command source **6**

5 A generalized counter seed and the actual blank trace **10**

6 A second physical allocation with a different axiom sampler **11**

7 Verification layers and reproducibility **13**

A Complete canonical transition table **15**

1 Result, distinction, and scope

The machine uses one bi-infinite binary tape and moves left or right at every step. It has 215 working states and 430 transition rows. Including the halt label gives 216 labels, not 216 working states. All working states are reachable in the directed transition graph; this is not a claim that every state occurs on the blank computation.

The desired conclusion is

$$\mathrm{ZFC} \vdash \big(\mathrm{Halts}(M, \mathrm{blank}) \leftrightarrow \exists p\ \mathrm{Proof}_{\mathrm{ZFC}}(p, \bot)\big).$$

Equivalently, ZFC proves that blank nonhalting of M is equivalent to Con(ZFC). No assertion of actual nonhalting, consistency of ZFC, minimality, or priority over every unpublished construction is made.

1.1 What is materially different?

The primary reference is the earlier composed-command/partial-selector K215 construction. A second reference permutes the DROP and NIP tags. A recent public variant exchanges two helper roles, at native labels 11 and 134. The new table is not isomorphic to any of these as a transition graph preserving the input symbol, written symbol, and move direction, even when the initial state is not distinguished. Joint color refinement distinguishes the respective graph multisets in 3, 3, and 2 rounds. This is a finite certificate of nonisomorphism, not a general graph-isomorphism heuristic: unequal refinement multisets preclude an isomorphism.

Against the primary reference, 17 retained transition rows change and 4,519 of the 4,751 primitive addresses change. Against the other two references, the literal native-label differences comprise 33 and 22 rows respectively. Native-label differences alone would not prove material novelty; the graph certificate and different blank computation provide the stronger evidence.

The old primary reaches a clean $(N, D) = (4, 0)$ source root after 95,714 steps. The new table reaches $(N, D) = (4, 1)$ after 716,307 steps. These are different physical computations and different proof boundaries. The old initializer certificate cannot simply be copied to the new table.

The logical calculus and abstract register program are unchanged. In particular, this is not a new axiomatization of set theory, a new command calculus, or a further reduction in state count. The change is a different allocation of an equivalent source encoding, plus a proof that a less restrictive initialized counter is sufficient.

1.2 Pinned artifacts

The native transition table has SHA-256

```
f5acfbb2e3d0e2b61350454a098a217d3457710311e9dbf02135098fdbe8d36d
```

The normalized discovery table has SHA-256

```
6aab0397336a198c8fd765803864a69134508175e637d563d6bea8aabbe7b416
```

The exact indented source-site file has SHA-256

```
704a0449c4164c39464f714e08c435d656f96c024e336ff27d091166772816fe
```

These hashes identify literal files, including their serialization. The native, normalized, and dense canonical tables are explicitly bijective relabelings of this one construction. The machine definition is the table, not its name or these state-label conventions.

2 Equivalent source encoding and physical verification

The source has 4,751 primitive occurrences and 202 distinct subtrees. Reassociation changes source addresses and the corresponding physical routing, while preserving the literal left-to-right operation/operand word and both guarded successor choices at every conditional primitive. The first 37 primitive addresses remain fixed. The maximum address depth increases from 47 to 49. The number of addresses containing a run of six zero bits decreases from 37 to 33. This last quantity counts addresses, not states or individual zero runs, and is not claimed to be a complexity lower bound.

Lemma 2.1 (All-natural source correspondence)**.** *Relate the old and new source configurations when their primitive ordinals and all nine natural-valued registers agree. This relation is a strong step-for-step bisimulation, preserving halt.*

Proof. At each ordinal the primitive operation and operand are identical. Therefore its arithmetic update and guard outcome agree on every natural store. The complete guarded ordinal control graphs agree, so the next ordinal or halting result agrees as well. Induction on execution length gives the claim. This argument has no bounded-register assumption and no formula-decoding premise. □

Source correspondence does not by itself prove that the reallocated binary table implements that source. A separate symbolic runtime check treats every primitive occurrence, splitting decrement cases according to zero or positive input. The fresh check proves all 6,176 cases and uses 1,753,058 symbolic rules. The minimum constant term in the certified affine transition duration is 6; all duration coefficients are nonnegative. Thus every source primitive corresponds to a positive finite binary execution, ruling out a zero-time simulation loophole.

The representation contracts allow arbitrary natural primary registers, an arbitrary natural padding gap, and a nonempty finite spare list, together with a completely zero exterior. Scan and list lemmas are inductive, not bounded test traces. Address decoding, both guarded exits, the represented output values, and the tape outside the represented frame remain separate proof obligations. The supplied proof machinery rechecks these rather than inferring them from the source DAG size.

The source/calculus bridge is inherited from the earlier composed K215 and its local-input K216 parent. The following sections reproduce the numerical calculus and explicit command compiler to make clear exactly which mathematics is retained. The final seed theorem extends their stated initialization boundary; it does not silently identify the new counter value with zero.

3 The numerical calculus and its consistency calibration

The present construction uses the dominated-parameter calculus supplied in `K218_Dominated_Rho8_2026-10-20.zip`. This is a recent input to the repository, distinct from its primary, unchanged-calculus K218 table. Its logical theorem is inherited through the 217-state parent. The present construction retains that numerical calculus and changes the physical source encoding and blank initialization. This distinction is essential for identifying which argument supplies each part of the result.

3.1 Raw numerical operations

For natural numbers put

$$P(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \qquad I(a,b) = 4P(a,b), \qquad a \to b := I(a,b).$$

The remaining constructors are

$$R(a,b) = 4P(a,b) + 1, \quad Q_a b = 4P(a,b) + 2, \quad E(a,b) = 4P(a,b) + 3.$$

Raw truth is $T = 0$ and the refutation target is $F = 1 = R(0,0)$. Write $NX = I(X,F)$ and $\exists_a X = N(Q_a(NX))$. In particular, $I(0,0) = 0$: zero is simultaneously the designated initial theorem and an implication code. All quantified schema parameters below range over *every natural number*. No test that a number decodes as a conventional formula is introduced.

Let $\mathcal{C}$ be the accepted native calculus, called `C222` in the inherited interfaces and used by K219. It has truth, numerical modus ponens, and theorem-only generalization:

$$\frac{\mathcal{C} \vdash X \quad \mathcal{C} \vdash I(X,Y)}{\mathcal{C} \vdash Y}, \qquad \frac{\mathcal{C} \vdash X}{\mathcal{C} \vdash Q_a X}.$$

Its eight logical rows comprise the following seven retained families and the unrestricted family L displayed below:

$$\begin{aligned}
\rho_1(a,b) &= \exists_a E(a,b),\\
\rho_2(a,b,c) &= Q_a(b \to c) \to (Q_a b \to Q_a c),\\
\rho_3(a,b,c) &= E(a,b) \to (R(c,a) \to R(c,b)),\\
\rho_4(a,b,c) &= Q_a Q_b c \to Q_b Q_a c,\\
\rho_5(a,b,c) &= E(c,a) \to (R(c,b) \to R(a,b)),\\
\rho_6(a,b) &= (\exists_a Q_a b) \to b,\\
G(a,b,c) &= E(c,a) \to \big(E(c,b) \to Q_{R(a,b)} E(a,b)\big),\\
L(A,B,C,U) &= ((C \to U) \to A) \to ((A \to C) \to (B \to C)).
\end{aligned}$$

The accepted set-theoretic information remains in the literal gated seed bundle. To specify this inherited object, define

$$\begin{aligned}
B(b,k,m) &= Q_b\big(m \to NQ_b(m \to NR(b,k))\big),\\
S(a,b,k) &= Q_k\big(R(k,a) \to R(k,b)\big),\\
C(a,b,k,m) &= Q_b\big(R(b,k) \to NQ_a(m \to NQ_a(S(a,b,k) \to B(b,k,m)))\big),\\
J(a,b,c) &= Q_c\big(C(a,b,c,c) \to NB(b,c,c)\big),\\
V(a,b,c) &= R(a,b) \to Q_c R(a,b),\\
\mathrm{Bundle}(a,b,c) &= N\big(V(a,b,c) \to J(a,b,c)\big).
\end{aligned}$$

A bundle is an axiom when $c \neq a$ and $c \neq b$. Both occurrences of c as matrix code and private label are retained literally. We neither remove these gates nor replace the bundle by an informal set-theoretic analogue. The source returns truth instead when a gate fails.

The established native calibration supplies finite-refutation transformations between $\mathcal{C} \vdash F$ and ZFC inconsistency. That inherited result is a premise of the present construction. It is supported by the repository's native-calculus developments and their connection to the compact set theory underlying K230; it is not proved anew by a transition-table simulation.

For provenance, the dominated package pins its K219 inputs to ProveIt revision `f8ec6158f3bdc8 85139ee395ae34fe44c3251b9f` and includes its exact native interfaces and finite logic certificate. Earlier proof-assistant endpoints concern their own exact tables. The native-calculus bridges, dominated theorem and present source/runtime arguments must all be included to reach the new table. No new exact-215 proof-assistant endpoint or internal PA proof object is asserted here.

3.2 Why a dominated fourth parameter is sufficient

Define $\mathcal{D}$ by retaining all of the preceding clauses, except that its $L(A, B, C, U)$ axiom requires

$$U \leq A. \tag{D}$$

This is an ordinary inequality between natural codes, not a syntactic freshness test and not an inequality of semantic truth values.

Theorem 3.1 (Inherited dominated-parameter theorem)**.** *For every natural code n,*

$$\mathcal{D} \vdash n \quad \Longleftrightarrow \quad \mathcal{C} \vdash n.$$

The proof is uniform in arbitrary native codes, including zero and all degenerate constructor coincidences.

The easy direction follows because every dominated instance is already an unrestricted instance. For the converse, the inherited finite certificate derives the entire unrestricted L schema from truth, modus ponens, and dominated instances alone. Here are the arithmetic and proof-theoretic facts that make that certificate meaningful.

First, $P(a, b) \geq a, b$ for all naturals. If $a + b = s \geq 1$, then $s(s + 1)/2 \geq s$, and the case $s = 0$ is immediate. Consequently $I(a, b) \geq a, b$. If a schematic term U is literally a subterm of an implication term A, then $U \leq A$ under every numerical assignment. The same conclusion holds whenever $U = T$. This remains true when some implications evaluate to zero. The finite checker uses precisely these sufficient conditions for all dominated leaves.

Second, every certificate node denotes a universally quantified *numerical* statement. Replacing one of its metavariables by an implication term is therefore universal instantiation. It does not assume a general substitution rule inside the object calculus, and it does not rename the numerical labels of a previously decoded atom. At an MP node, equality of the instantiated antecedent and minor is checked exactly. The sole native normalization is $I(T, T) = T$.

The unrestricted-row certificate has the following exact inventory:

Item	Count
Proof nodes	3,728
Shared term nodes	5,160
Dominated-L leaves	1,260
Truth leaves	1
MP steps with explicit substitutions	55
Direct universal-instantiation steps	410
Direct MP steps	2,002
Maximum checked subterm depth	4

These are finite proof objects, not samples of numerical assignments. The implication-subterm inequality proves each axiom side condition for all assignments; induction through the checked proof proves the schematic result for all assignments.

The certificate first obtains identity, weakening K, Peirce's law, composition, and distribution S:

$$\begin{aligned}
&K : p \to (q \to p),\\
&\mathrm{P} : ((p \to q) \to p) \to p,\\
&\mathrm{B} : (p \to q) \to ((q \to r) \to (p \to r)),\\
&S : (p \to (q \to r)) \to ((p \to q) \to (p \to r)).
\end{aligned}$$

A key intermediate theorem is

$$(p \to q) \to (((p \to r) \to p) \to q).$$

The successful inherited proof of that theorem adds the dominated context $A = ((z \to U) \to v) \to w$; the fourth argument lies three implication edges inside the first. The rest of the fixed derivation reaches unrestricted L. The complete certificate and its checkers are included with the inherited dependency, so the large intermediate formulas need not be expanded into the body of this article.

For clarity, the final passage from K, S, P to L has a short ordinary explanation. Temporarily assume $H = (C \to U) \to A$ and $J = A \to C$. Under the additional assumption $C \to U$, two MPs give A and then C. The deduction transformation justified by K, S gives $(C \to U) \to C$. Peirce's law at C, U gives C. Weakening gives $B \to C$, and discharging J, H yields exactly $L(A, B, C, U)$. No generalization is used under these temporary assumptions. The supplied certificate expands this deduction argument into explicit schema instances and MP steps.

Replacing each unrestricted-L leaf of a finite $\mathcal{C}$ proof by this fixed derived proof gives a finite $\mathcal{D}$ proof of the same code. All other axioms and both rules are retained literally, proving the theorem. Combining it with the inherited native calibration gives

$$\mathcal{D} \vdash 1 \quad \Longleftrightarrow \quad \text{there is a finite ZFC refutation.} \tag{C1}$$

4 Inherited all-natural calibration of the local-input command source

This section reproduces, with attribution, the explicit ordinary calibration argument supplied with `K216_Permuted_Commands_2026-10-21.zip`, with attribution to that construction. Its source hashes and selector paths identify the parent reference trees, not the new reparenthesized source. The exact all-natural source correspondence in Section 2 transports this theorem to the present source without changing any arithmetic command or candidate encoding.

The inherited source-level transformation is a permutation of the three small command selectors. The new construction keeps these selectors; its additional initialization theorem is proved below. The arithmetic commands and the dominated axiom family are unchanged. This section proves that the permutation preserves the full consistency calibration, including raw candidate numbers, the zero alias, arbitrary formula codes, and the actual source seed with next-candidate register 4. It does not infer a binary runtime theorem from a source subtree count; the binary runtime and blank-tape certificates are separate obligations.

4.1 Exact finite source identity

Write

$$\mathsf{P}(a, b) = \frac{(a+b)(a+b+1)}{2} + b, \qquad \mathsf{I}(a, b) = 4\mathsf{P}(a, b), \qquad \mathsf{Q}_i a = 4\mathsf{P}(i, a) + 2.$$

The native truth and falsity codes are respectively 0 and 1. In particular, $\mathsf{I}(0, 0) = 0$. Let $\mathcal{D}$ be the dominated numerical calculus of the preceding dominated-rho8 construction: truth, numerical modus ponens, theorem-only generalization, the seven retained logical rows, the gated theory bundle, and

$$L(A, B, C, U) = \mathsf{I}(\mathsf{I}(\mathsf{I}(C, U), A), \mathsf{I}(\mathsf{I}(A, C), \mathsf{I}(B, C))), \qquad U \le A.$$

The inherited finite propositional certificate proves equality of the $\mathcal{D}$ and K219 theorem sets for *every natural code*. Together with the inherited numerical-calculus-to-ZFC calibration, it gives

$$\mathcal{D} \vdash 1 \quad \Longleftrightarrow \quad \exists p \ \mathrm{Proof}_{\mathrm{ZFC}}(p, \bot). \tag{1}$$

No well-formedness or successful formula-decoding test is inserted into this equivalence.

The source checker first reconstructs the 205-subtree noncommon-input reference directly from the pinned K219 source. Its independently implemented structured-control and address-counter interpreters agree on every primitive and every branch outcome. The resulting guarded ordinal control graph is exactly that of the frozen 202-subtree source with SHA-256

`abb7229877759170b61dfcfe21ca0889b3df2f91f4215383817a125eb0f8996c.`

Thus the frozen source has the arithmetic command semantics proved in the dominated-rho8 report, with counter/cache field order YX.

In this frozen source, the MP, DROP, and NIP bodies occur at paths 111, 110101, and 110100101, respectively. They contain 106, 26, and 26 primitive occurrences. Each has zero escaping break and continuation demand. NIP contains the halting primitive; all its nonhalting exits are normal returns. The branch-skipping instructions outside the three bodies remain fixed when the bodies are permuted. The checker traces their ordinary continuations: the last body returns directly to source ordinal 0, the middle body returns through the unconditional break at 4724, and the innermost body returns through the breaks at 4697 and 4724. Each route ends at ordinal 0 without another arithmetic command.

For the source denoted 210, put NIP, DROP, and MP in these three selector positions. For 120, put DROP, NIP, and MP there instead. The exact selector partitions and wire formats are

	selector 0	selector 1	selector 2	selector $c+3$
210	NIP	DROP	MP	$\mathsf{AX}(c;x,y)$
120	DROP	NIP	MP	$\mathsf{AX}(c;x,y)$

and

$$f_{210}(\mathsf{NIP})=[0],\qquad f_{210}(\mathsf{DROP})=[1],\qquad f_{210}(\mathsf{MP}(x))=[2,x],$$
$$f_{120}(\mathsf{NIP})=[1],\qquad f_{120}(\mathsf{DROP})=[0],\qquad f_{120}(\mathsf{MP}(x))=[2,x],$$
$$f_{\pi}(\mathsf{AX}(c;x,y))=[c+3,y,x]\qquad(\pi=210,120).$$

The semicolon in $\mathsf{AX}(c;x,y)$ separates the selector parameter from the counter increment x and cache increment y; it does not denote wire order.

The independent checker proves these partitions directly from the three selector decrement tests, at primitive ordinals $23,24,25$. In the 210 source, their zero branches lead to NIP at ordinal 4725, DROP at 4698, and MP at 4591. Three positive outcomes enter AX at ordinal 26, with residual selector c. This last case is the symbolic case of every input selector at least 3, not a finite sampling of large selectors.

Both sources have 202 distinct subtrees, 4,751 primitive occurrences, 6,176 guarded primitive cases, maximum address depth 44, and maximum zero run 6. All 4,593 primitive occurrences exterior to the replaced bodies, including the entire AX subtree, remain literal. The checker also verifies that each candidate tree is *exactly* the indicated permutation of the frozen tree. The candidate source hashes are

```
210:  8b5dc46f10d927b2db5e4bcb522d482779ec61c72c82717d7bf12fca3550a3bc
120:  ee1f7c7cb216a80315449371b040ee094728a655439954225ed4be4635296680.
```

4.2 Command equations and soundness for raw codes

At a command boundary write the retained tuple as (t,w,d,h): top, encoded stack, additive counter, and cached formula. The notation $[x_1,\dots,x_k;w]$ means right-nested pairing, and

$\mathrm{unpair}(0) = (0, 0)$. The exact command equations, independent of which small tag names the command, are

$$\mathsf{MP}(x)(t, w, d, h) = \begin{cases} (d + x, \mathsf{P}(0, w), d + x, 0), & \mathsf{I}(t, d + x) = h, \\ (0, w, d + x, 0), & \mathsf{I}(t, d + x) \neq h; \end{cases}$$

$$\mathsf{DROP}(t, w, d, h) = (h, \mathsf{P}(t, w), d, h),$$

$$\mathsf{NIP}(t, w, d, h) = \begin{cases} \mathsf{halt}, & h = 1, \\ (t, w', d, h'), & (h', w') = \mathrm{unpair}(w),\ h \neq 1. \end{cases}$$

For $\mathsf{AX}(c; x, y)$ put

$$U = d, \qquad A = d + x, \qquad B = h + y, \qquad C = c,$$

and let $G(A, B, c)$ be zero when $c = A$ or $c = B$, and the retained theory bundle otherwise. In the inherited stack order its output is

$$t' = G(A, B, c), \qquad d' = 0, \qquad h' = 0,$$
$$w' = [R_1(A, B, c), \ldots, R_7(A, B, c), L(A, B, c, U), \mathsf{Q}_U h, t; w].$$

Here the h in $\mathsf{Q}_U h$ is the incoming cache, before adding y. The bound $U \leq A$ holds for every raw payload because $A = d + x$.

A body with no escaping break or continuation has the same effect at any of the three command positions. The branch wrappers route every normal return back to the command loop. Consequently the equations above hold for the permuted sources on every natural input tuple, including payloads arising from incomplete or otherwise unintended candidate encodings.

Soundness invariant. If t, h, and every successive head of w are $\mathcal{D}$-theorems, every nonhalting command preserves that property. A successful MP is exact numerical modus ponens; a failed MP inserts truth 0. DROP and NIP only move theorem codes. AX inserts retained axioms, a valid dominated-row instance, a generalization of a theorem, and either a permitted bundle or truth. The only halt is the NIP test $h = 1$. Therefore halting from any such safe seed implies $\mathcal{D} \vdash 1$. This argument quantifies over all natural codes and does not discard "junk" candidates.

4.3 An explicit compiler retained under the tag change

For completeness, a strong block $\mathcal{B}_X$ must, from every $(t, w, 0, 0)$, either halt or return $(X, w, 0, 0)$. The command lists below use command *names*; their concrete wire encodings use f_{210} or f_{120}. Concatenation denotes sequential execution, and a superscript on a command denotes repetition.

Define

$$\mathsf{Discard} = [\mathsf{MP}(1), \mathsf{AX}(1; 0, 0)]\mathsf{NIP}^{10}, \qquad J(u) = \begin{cases} [], & u = 0, \\ [\mathsf{MP}(u)], & u > 0. \end{cases}$$

The first MP fails because $\mathsf{I}(t, 1) > 0$, whereas the incoming cache is zero. The following AX has $U = A = c = 1$ and a zero gate. Its nine newly generated codes are all different from 1, and its saved old top is zero. Hence ten NIPs return exactly $(0, w, 0, 0)$. The block $J(u)$ maps $(0, w, 0, 0)$ to $(0, w, u, 0)$: if $u > 0$, $\mathsf{I}(0, u) > 0$ forces failure. Omitting MP when $u = 0$ avoids the extra stack zero that a successful $\mathsf{MP}(0)$ would insert.

Truth uses $\mathsf{Discard}$. A dominated-row leaf with $U \leq A$ uses

$$\mathsf{Discard}\ J(U)\ [\mathsf{AX}(c; A - U, B)]\ \mathsf{NIP}^8\ [\mathsf{DROP}]\ \mathsf{NIP}^3.$$

For retained row $R_j(A, B, c)$, where $1 \leq j \leq 7$, use

$$\mathsf{Discard}\ [\mathsf{AX}(c; A, B)]\ \mathsf{NIP}^j\ [\mathsf{DROP}]\ \mathsf{NIP}^{11-j}.$$

For a permitted bundle use $\mathsf{Discard}[\mathsf{AX}(c; A, B)]\mathsf{NIP}^{10}$. These suffixes select the indicated top or stack entry and restore the underlying stack and zero cache. After expanding the negation and existential abbreviations, all eight row codes are implications, and $\mathsf{Q}_U h \equiv 2 \pmod 4$, so none of these generated stack codes is falsity 1.

Given a block for X, a block for $\mathsf{Q}_i X$ is

$$\mathcal{B}_X\ [\mathsf{DROP}]\ J(i)\ [\mathsf{NIP}, \mathsf{AX}(i; 0, 0)]\ \mathsf{NIP}^9\ [\mathsf{DROP}, \mathsf{NIP}, \mathsf{NIP}].$$

After DROP and $J(i)$, the current top and cache are zero, the counter is i, and X is the first stack head. NIP places X in the cache. AX then has $U = A = c = i$, so its top gate is zero and its ninth generated stack entry is exactly $\mathsf{Q}_i X$. The selection suffix returns that code with the original stack and zero counter and cache.

Given blocks for X and $\mathsf{I}(X, Y)$, use

$$\mathcal{B}_X\ [\mathsf{DROP}]\ \mathcal{B}_{\mathsf{I}(X,Y)}\ [\mathsf{NIP}, \mathsf{DROP}, \mathsf{NIP}, \mathsf{MP}(Y), \mathsf{AX}(Y; 0, 0)]$$
$$\mathsf{NIP}^{10}\ [\mathsf{DROP}, \mathsf{NIP}, \mathsf{NIP}].$$

Unless an earlier NIP has halted, the two premises reach the top and cache in the required order, and MP succeeds. The reset AX has $U = A = c = Y$ and a zero gate. The final selection returns Y with the original stack and zero counter and cache, or halts if a tested cache is already 1.

Induction on a finite $\mathcal{D}$ derivation proves the strong block contract for every derived natural code. The construction is primitive recursive on the derivation. The new selectors alter only the wire names of these commands, so every calculation above applies verbatim to both permuted sources.

4.4 Exact packing and cofinality from the actual cutoff

For a finite field list fs and tail r, write $E(fs, r)$ for its right-folded Cantor pairing. Command tails are encoded by

$$\mathrm{enc}_\pi([]) = 0, \qquad \mathrm{enc}_\pi(k :: L) = 1 + E(f_\pi(k), \mathrm{enc}_\pi(L)).$$

A newly enumerated candidate is loaded directly from its number. A subsequent command is decoded from one less than the current positive proof-list value. Thus the number for a whole first candidate $k :: L$ is $E(f_\pi(k), \mathrm{enc}_\pi(L))$, without an outer successor. This distinction is essential when NIP has tag zero.

Every field reader returns a tail no greater than its input. Therefore one command starting from a positive proof-list value v returns a proof-list value at most $v - 1$, unless it halts. All arithmetic macros terminate on all natural inputs. The full macro contracts also restore selector $A = 0$ and scratches $X = Y = 0$ at every nonhalting command return; they preserve N outside candidate fetch and leave L equal to the unconsumed unpairing remainder. At fresh fetch the source loads N as the new candidate and increments N exactly once. Every nonhalting candidate therefore drains after finitely many commands; the search then proceeds to the next candidate. In particular, zero selectors or zero field tails do not cause an omitted case or a nonhalting inner computation.

Suppose $\mathcal{D} \vdash 1$, let $\mathcal{B}_1$ be its compiled block, and put

$$S = [\mathsf{DROP}, \mathsf{NIP}, \mathsf{NIP}], \qquad R = \mathrm{enc}_\pi(\mathcal{B}_1 S).$$

For any natural padding value Y, prepend $\mathrm{Start}_Y = \mathsf{AX}(0; 1, Y)$. It sets the counter and cache to zero from every incoming retained tuple. Thus $[\mathrm{Start}_Y]\mathcal{B}_1 S$ halts from every incoming tuple: the block either halts, or returns top 1 and zero cache, and S then puts 1 in the cache and tests it. Its exact first-candidate number is

$$N_Y = \mathsf{P}(3, \mathsf{P}(Y, \mathsf{P}(1, R))) \geq Y. \tag{2}$$

The inequality follows from the coordinate inequalities for Cantor pairing. This gives a cofinal family of successful candidates for both permutations. Choose $Y \geq 4$. The successful candidate N_Y is at or beyond the actual initial next-candidate value 4. Finite draining of earlier candidates guarantees that the search reaches it unless a halt has already occurred. More generally the same proof works beyond every natural cutoff, and after every finite initial partial candidate.

Source calibration theorem. For either source permutation, started at its genuine source root with register tuple

$$(N, T, W, A, D, H, X, Y, L) = (4, 0, 0, 0, 0, 0, 0, 0, 0),$$

ZFC proves

$$\mathrm{Halt}_{\mathrm{source}} \iff \mathcal{D} \vdash 1 \iff \exists p\ \mathrm{Proof}_{\mathrm{ZFC}}(p, \bot).$$

Proof. The zero top, zero cache, and all heads of the zero stack are truth theorems. The soundness invariant proves the forward implication. The explicit compiler, cofinal family (2), and draining argument prove the reverse implication. Equation (1) gives the last equivalence. The argument uses finite source identities, primitive-recursive proof and word transformations, and induction on natural numbers and finite derivations. It is formalizable in ZFC without assuming Con(ZFC).

Combining this theorem with a separate exact binary runtime simulation and a blank-tape trace to the displayed represented source root yields

$$\mathrm{ZFC} \vdash (\text{the corresponding binary machine does not halt on blank tape}) \longleftrightarrow \mathrm{Con(ZFC)}.$$

The trace's tape header, head position, gap and spare-word requirements belong to that binary certificate; they are not consequences of the source calibration alone.

5 A generalized counter seed and the actual blank trace

Write the nine logical registers in the order $(N, T, W, A, D, H, X, Y, L)$. Here N is the next candidate index, T the current theorem, W the encoded stack, A the command selector, D the additive counter, H the theorem cache, X, Y scratches, and L the remaining proof-list code.

Theorem 5.1 (Arbitrary-counter root calibration)**.** *At a genuine source root, let $N = n$ and $D = d$ be arbitrary naturals, let $A = X = Y = L = 0$, and suppose T, H and all stack heads of W are numerical theorems. Then source halting is equivalent to numerical inconsistency. In particular the theorem applies to $n = 4, d = 1, T = H = W = 0$.*

Proof. Soundness does not require $D = 0$. In an AX command the old counter is $U = d$ and its new parameter is $A = d + x$, so the dominated-row requirement $U \leq A$ holds for every natural payload x. The seven other axiom rows, theorem-only generalization, and the gated literal bundle retain their old soundness arguments. MP is numerical detachment when its equality test succeeds and produces truth otherwise. DROP and NIP only move theorem codes. The sole halting test is cached falsity $H = 1$. Thus a halt implies inconsistency even when the initial counter is nonzero.

For the converse, take the strong compiled block $\mathcal{B}_1$ for a numerical derivation of falsity and append $S = [\mathsf{DROP}, \mathsf{NIP}, \mathsf{NIP}]$. The strong compiler works from every retained top and stack once $D = H = 0$; it either halts or returns $(T, W, D, H) = (1, w, 0, 0)$ with the original stack w. The suffix then loads and tests falsity.

For each natural $y \geq 1$, prefix this list by $\mathsf{Start}_y = \mathsf{AX}(0; 1, y)$. For arbitrary incoming d, h, this command has $U = d$, $A = d + 1$, $B = h + y$, and $C = 0$. Both gate inequalities hold because $A, B > 0$. More importantly, irrespective of the incoming top, stack, or counter, the command sets $D = H = 0$. Its new top and stack need not be prescribed: the strong block explicitly quantifies over arbitrary such data. Any earlier halt is already sufficient for the completeness direction.

Put $R = \mathrm{enc}_{210}(\mathcal{B}_1 S)$. The exact first-candidate integer for the prefixed list is

$$n_y = P(3, P(y, P(1, R))) \geq y.$$

There is no outer successor on a first candidate. Subsequent positive proof-list entries have the inherited successor convention. Choose $y \geq \max(1, n)$. Every preceding candidate either halts or drains in finite time: its first command performs finite arithmetic; each subsequent command strictly decreases its positive list remainder. The scheduler restores its selector and scratches and advances N. Consequently it reaches n_y unless a halt has already occurred. That candidate resets the counter/cache, executes the compiled refutation, and halts. This proof does not assume a bound on the unknown counter carried into the successful candidate. □

5.1 Literal blank initialization

Exact simulation from the defined blank configuration reaches, after 716,307 transitions,

$$q = 84, \quad h = -9, \quad g = 152, \qquad (N, T, W, A, D, H, X, Y, L) = (4, 0, 0, 0, 1, 0, 0, 0, 0),$$

in native labels and absolute tape coordinates. The spare list consists of 185 ones. The complete finite set of nonzero tape cells is supplied as a receipt and is checked, not merely the nine decoded numbers.

The leading one is exactly at $h - 1 = -10$. The next one is at $h + 64 + g = 207$, so all 64 program-counter cells are zero. This is a genuine source root rather than an interior instruction with a conveniently decodable tail. The decoder checks the full unary-register frame, spare list, and zero exterior. No halt occurs in the verified prefix.

The zero current theorem and cache are truth; the zero stack has only truth under the inherited zero-pairing alias. The generalized theorem applies with $n = 4, d = 1$. Combining it with source bisimulation and the positive unbounded binary simulation proves the claimed blank-start consistency equivalence. All arguments concern finite tables, natural-number induction, primitive-recursive proof transformations, and finite derivations, and are formalizable in ZFC without assuming its consistency.

A separate reachability check contains the new counter-one seed in the old closed abstract invariant: its ternary class is 83 and its four-class index is 259. A fresh ternary closure reproduces the same bitmap. This is a useful cross-check, but it is not used as a substitute for either the arbitrary-counter theorem or the all-natural binary runtime proof.

6 A second physical allocation with a different axiom sampler

A complementary alternative changes the *source command effect*, rather than only its encoding. It emits $\rho_6(A, B)$ in place of $\rho_6(A, C)$, where

$$\rho_6(a, b) = (\exists_a Q_a b) \to b.$$

The universal numerical calculus is still the same: only the way a command supplies an instance is changed. This sampler idea and its composed source are already recorded in public intake 199 at ProveIt commit `7de71bd17b85db67529e8f10083c066ab0ed0c56`. Our contribution is a distinct physical allocation and its completed verification, not priority for the sampler idea.

Relative to the primary *earlier* K215 table, not the counter-one table of this report, the new native table changes exactly two rows:

row	old action	new action
199/1	$1R166$	$1R12$
206/0	$0R186$	$0R76$

No state is added or removed. Its native table SHA-256 is

```
a96732bd55c9412cbfd4bc3019200f364b3e3169f58e9a495dd98b072fb2d648
```

Its canonical table SHA-256 is

```
bfa5ad02c53193656ed135588416bd05bdc72ffa2de92e3f6ed13556268df0d4
```

The four changed primitive operands are at zero-based ordinals $959, 964, 1137, 1142$, changing register index 3 to 5. The source interval $[888, 1171)$ is the 283-leaf sixth-axiom emitter. The tree retains 202 distinct subtrees and 4,751 leaves. Its exact source SHA-256 is

```
19da13e482ea0f3d57432aad852bdf2448093e58a6e3b3873ac744eae7440693
```

6.1 All-natural sampler theorem

Conjugate the old emitter by swapping its parameter registers 3 and 5. The literal interval changes only the four listed CI/CD operands; all guarded successors remain fixed. The old emitter preserves its parameter registers and clears its scratches. Consequently the conjugate restores the same full input parameter store, constructs $\rho_6(A, B)$, and pushes the old current theorem on the same underlying stack. This is a stepwise all-natural register conjugacy, not an assumption that arbitrary derived formulas may be renamed or that a finite test set exhausts numerical inputs.

Every emitted code is therefore an instance of the same universally quantified native axiom. Other axiom emitters, the dominated fourth parameter, theorem-only generalization, and the literal gated bundle remain unchanged. The old theoremhood invariant still proves halting soundness for all raw codes.

Completeness needs an adjusted strong compiler for the sixth row. For arbitrary a, b, use

$$\mathsf{Discard}; \mathsf{AX}(a; a, b); \mathsf{NIP}^6; \mathsf{DROP}; \mathsf{NIP}^5.$$

After Discard, the top and counter/cache are zero and the underlying stack is the original w. This AX has $U = 0, A = a, B = b, C = a$ and wire fields $[a + 3, b, a]$. The bundle gate fails because $C = A$, so the new top is zero. After six pops, $\rho_6(a, b)$ is cached. DROP makes it current and pushes one extra zero. Five more pops remove that extra zero and the four remaining entries, ending with the saved old top zero in cache. The underlying stack is restored and $D = H = 0$.

All intermediate axiom entries are implications, the generalized cache has tag 2, and the saved top is zero, so none is the refutation code 1. The other strong row blocks retain their selection positions and cleanup counts. Their unused sixth entry has changed but remains a nonfalsity theorem. MP, GEN, the strong compiler induction, exact candidate packing, and cofinal refutation padding are unchanged. This proves consistency equivalence for the new source.

6.2 Physical distinction and initialization

Reconstructing the decoder from the changed source and table is essential. An earlier inherited-allocation attempt kept stale roles and failed finite tests; that failure was not a proof that the two-row construction needed an extra state.

The new table is not an initial-state-preserving labelled-graph relabeling of either the old primary table or the public intake-199 allocation. The latter differs at 62 literal state/symbol keys; a forced rooted mapping also reaches a written-symbol/direction conflict, so this distinction does not rely on native label choices alone.

Its blank prefix is unchanged: neither modified row is executed before the certified root. Independent literal replay reaches the genuine source root after 95,714 transitions, with head -8, gap 1, 40 all-one spares, and $(N, T, W, A, D, H, X, Y, L) = (4, 0, 0, 0, 0, 0, 0, 0, 0)$. The complete sparse tape and dense relabeling are checked against the prefix receipt. The separate fresh arbitrary-natural runtime verification passes all 6,176 guarded cases, using 1,699,838 symbolic rules with minimum affine duration 6 and coverage of all 430 rows. Its exact table bindings and certificates are supplied with the secondary artifacts. The two machines in this report have independent physical certificates; their common state count is not used to transfer one proof to the other.

7 Verification layers and reproducibility

The archive separates the following claims. Passing one layer is not treated as proving the next.

1. **Literal machine.** Parse both transitions of every working state, verify all targets, count 215 working states and 430 rows, and check the native/normalized/canonical bijections. The canonical table in the appendix is generated from the machine file.
2. **Source identity.** Check the entire primitive word and complete guarded ordinal control graph against the calibrated primary source. This proves the all-natural source bisimulation, not binary execution.
3. **Binary runtime.** Recheck 6,176 guarded symbolic cases against the exact table and source. These use inductive scan/list lemmas and positive affine durations, rather than a fixed cutoff on register values.
4. **Blank prefix.** Independently replay all 716,307 transitions from zero tape and compare the entire final sparse tape. Decode its full frame and zero program counter; do not accept the first superficially well-formed interior frame.
5. **Source consistency.** Use the explicit numerical-calculus calibration, strong proof compiler, finite draining, and the new arbitrary-counter root theorem. This is what connects physical halting with ZFC inconsistency.
6. **Distinctness.** Check the graph-refinement certificate and exact source/table differences against the three pinned baselines. This excludes a mere relabeling relative to those inputs, not every possible previously published construction.

The ordinary proof engine and its certificates are executable mathematical evidence. They are not a newly checked PA proof object, and this work does not supply a new exact-table Lean/Rocq endpoint. The inherited set-theoretic calibration is explicitly part of the provenance chain; this work does not claim to formalize it afresh. Runtime replay by the same checker is a fresh execution, not an independent second proof algorithm. The additional sparse-tape initializer and graph/source audits are independently implemented checks of their respective finite claims.

7.1 What did not work toward 214

A separate 214-state initialized core implements the original register source in all 6,176 unbounded primitive cases. It is *not* a blank-start construction: starting it from any of its 214 existing states on blank tape leads to a verified outward ray. Consequently it cannot be substituted for the present 215-state machine. Context-sensitive startup repair remains open.

Three further 214-state tables passed an initially too-small macro sample. Broader admitted inputs exposed incorrect arithmetic, including an actual blank-run mismatch for one table. Those tables are not certified alternatives and are not included among the claimed results. This experience motivates explicitly distinguishing finite search filters from unbounded contracts and from a changed source that would require its own consistency proof.

7.2 Provenance

The public project is `https://github.com/VladimirReshetnikov/ProveIt`, in `Computability/BusyBeaver/Research/ZFC/`. The numerical dominated-row recovery traces back to commit `7071e5041ee316d087661b658362feed159901ef`; the local-input permutation parent is commit `d59b4418821c598c4703f0a60b1f7cbae4f3bcf6`. The previous composed K215 packet supplies the immediate source/runtime reference and reproduced calibration sections. Its two state tables, and the later helper-role variant, are pinned in the comparison evidence. Attribution and inherited licenses accompany the code rather than being replaced by a claim that this is an independent new calculus.

A Complete canonical transition table

Start state 0, head coordinate 0, and an everywhere-zero tape are part of the definition. Working states are $0, \ldots, 214$; H denotes halt label 215, which has no outgoing transitions. An entry wDq means write w, move one cell in direction D, and enter state q. The appendix is generated from the supplied canonical table.

State	Read 0	Read 1	State	Read 0	Read 1
0	1R93	1R0	108	0R123	1R8
1	0L1	1L2	109	0R102	1R124
2	0L3	1L2	110	0R152	1R190
3	0L5	1L2	111	0L214	0L96
4	1R6	0L4	112	0R126	1R105
5	0L9	1L2	113	0R173	1R153
6	0R6	0R10	114	0R166	1R184
7	0R85	1R194	115	0R192	1R166
8	0R14	1R13	116	0R150	1R148
9	0L15	1L2	117	0R161	1R205
10	0L16	0R10	118	0L133	0L1
11	0L26	1R131	119	0R39	1R73
12	0R19	1R20	120	0R62	1R134
13	0L90	1L31	121	0R62	1R135
14	0L11	0R22	122	0R71	0L16
15	0L23	1L2	123	0R14	1R134
16	1L1	0L1	124	0R21	1R136
17	0R24	0R32	125	0R149	1R181
18	0R206	1R205	126	0R139	1L20
19	0R28	1R29	127	0R132	1R183
20	0R30	1R31	128	0R142	1R200
21	0R24	0R60	129	0R76	1R179
22	0R33	1R22	130	0R17	1R180
23	0L34	1L2	131	0R76	1R138
24	0L35	0R0	132	0R12	1R143
25	0R36	1R25	133	0R30	1R70
26	0R37	1R38	134	0R39	1L126
27	0R39	0L40	135	0R155	0L16
28	0R41	1R42	136	0R156	0L16
29	0R43	1R44	137	0R115	1R88
30	0R45	1R46	138	0R162	1R141
31	0R21	1R47	139	0R24	0R48
32	0R48	1R32	140	0R142	1R183
33	0R32	1R33	141	0R198	1R95
34	0L50	1L2	142	0R97	1R160
35	1R35	1L1	143	0R178	1R167
36	0R51	1R36	144	0R12	1R160
37	0R17	1R211	145	0R89	1L167
38	0R13	1R53	146	0R89	1R159
39	0L11	0R36	147	0L154	1L164
40	0L4	1L54	148	0R85	1R89
41	0R24	0R25	149	0R160	1R12
42	0R13	0L16	150	0R84	1R145
43	0R56	1R57	151	0R165	1R169
44	0R58	1R59	152	0L67	1L143
45	0R24	0R55	153	0R150	1R207
46	0R61	1R62	154	0R160	1R116
47	0R63	0L16	155	0R98	1R98
48	0R25	1R48	156	0R49	0R32
49	0L35	0R64	157	0R201	1R128
50	0L75	1L2	158	0L185	1R97
51	0R16	1R51	159	0R169	1R182
52	0R149	1R145	160	0R19	1R172

State	Read 0	Read 1	State	Read 0	Read 1
53	0R68	1R69	161	0R176	1R166
54	0L40	1L54	162	0R160	1R97
55	0R22	1R55	163	0R127	1R165
56	0L11	0R51	164	0R98	1R163
57	0R61	0L16	165	0L7	1R144
58	0R70	1R71	166	0L110	1R176
59	0R62	1R72	167	0R148	1R175
60	0R55	1R60	168	0R89	1R157
61	0R49	0R25	169	0R162	1R145
62	0L28	1L73	170	0R146	1R137
63	0L119	1L30	171	0R181	1R181
64	0R74	1R64	172	0R184	1R124
65	0R98	1R152	173	0R147	1R166
66	1R188	1L54	174	0R145	1R170
67	0R146	1R147	175	0R76	1R61
68	0R77	1RH	176	0R147	1R89
69	0L201	1R31	177	0R105	1R97
70	0R41	1R63	178	0R142	1R85
71	0L90	1R79	179	0R178	1R76
72	0R56	1L80	180	0R191	0L214
73	0R81	1R82	181	0R144	1L151
74	1R0	1R64	182	0R98	1R52
75	0R75	1R83	183	0R169	1R52
76	0L147	1L143	184	0R17	1L133
77	0R86	1R87	185	0R174	1R186
78	0R130	1R12	186	0R77	1R118
79	0R90	1L134	187	0R107	1R126
80	0L109	1R57	188	1L40	1L111
81	0R49	0R33	189	0R173	1R160
82	0R92	0L57	190	0R152	1R117
83	0R94	1R202	191	0R11	1R99
84	0R97	1R12	192	0R196	1R100
85	0R89	1R98	193	0R148	1R203
86	0R213	1R21	194	0R158	1R195
87	0R118	1R101	195	0R178	1R189
88	0R152	1R114	196	0R65	1R7
89	0R103	1R104	197	0R152	1R129
90	0R81	1R80	198	0R69	1R212
91	0R102	1R177	199	0R154	1R125
92	0R106	1R107	200	0R165	1R185
93	0R66	0R0	201	0R91	1R89
94	0R27	1R108	202	0R109	1R26
95	0R186	0L214	203	0R210	1R110
96	1L111	1L96	204	0R152	1R171
97	0R19	1R112	205	0R204	1R161
98	0R61	1R61	206	0R142	1R140
99	0R201	1R208	207	0R199	1R18
100	0R67	1R193	208	0R28	1R158
101	0R106	0L214	209	0R164	1R197
102	0R119	1R120	210	0R209	1R113
103	0R72	1R59	211	0R78	0L214
104	0R58	1R121	212	0R168	1R187
105	0R21	1R122	213	0R45	1R101
106	0R21	0L10	214	0L4	0L214
107	0R62	1R42			